# Modern Java ile

## Design Patterns'ın

## Fonksiyonel Evrimi

JavaDay İstanbul 2026

# Hakkımda

**Ümit Köse**

Software Developer

Türksat Uydu Haberleşme Kablo TV ve İşletme A.Ş.

SOSYAL MEDYA

- YouTube: @umiitkose
- X: @umiitkose
- GitHub: @umiitkose

REPO QR

Sunumdaki tum kodlar:

`github.com/umiitkose/javadayistanbul-functional-evolution-of-design-patterns-with-modern-Java`

# Gündem

# Gündem

**01** **Java 8 - Java 26**
Fonksiyonel Programlama Yolculuğu

# Gündem

**01** **Java 8 - Java 26**
Fonksiyonel Programlama Yolculuğu

**02** **Design Patterns**
Someone has already solved your problems.

# Gündem

**01** **Java 8 - Java 26**
Fonksiyonel Programlama Yolculuğu

**02** **Design Patterns**
Someone has already solved your problems.

**03** **4 Pattern**
Klasik OOP ↔ modern Java (records, lambda, kompozisyon)

**Pattern'ler**

- 🏗️ Builder → Immutability & Records
- 🎯 Strategy → Lambda & Functional Interfaces
- 🎨 Decorator → Function Composition
- 📋 Template Method → Higher-Order Functions

# Gündem

**01** Java 8 - Java 26
Fonksiyonel Programlama Yolculuğu

**02** Design Patterns
Someone has already solved your problems.

**03** 4 Pattern
Klasik OOP ↔ modern Java (records, lambda, kompozisyon)

**Pattern'ler**

- 🏗 Builder → Immutability & Records
- 🎯 Strategy → Lambda & Functional Interfaces
- 🎨 Decorator → Function Composition
- 📋 Template Method → Higher-Order Functions

**04** Genel Değerlendirme
Ne zaman OOP, ne zaman FP?

# Gündem

**01** **Java 8 - Java 26**
Fonksiyonel Programlama Yolculuğu

**02** **Design Patterns**
Someone has already solved your problems.

**03** **4 Pattern**
Klasik OOP ↔ modern Java (records, lambda, kompozisyon)

**Pattern'ler**

- 🏗️ Builder → Immutability & Records
- 🎯 Strategy → Lambda & Functional Interfaces
- 🎨 Decorator → Function Composition
- 📋 Template Method → Higher-Order Functions

**04** **Genel Değerlendirme**
Ne zaman OOP, ne zaman FP?

⏱️ 25 dakika • Java 8–26 odaklı

# Gündem

**01** **Java 8 - Java 26**
Fonksiyonel Programlama Yolculuğu

**02** **Design Patterns**
Someone has already solved your problems.

**03** **4 Pattern**
Klasik OOP ↔ modern Java (records, lambda, kompozisyon)

**Pattern'ler**

- 🏗️ Builder → Immutability & Records
- 🎯 Strategy → Lambda & Functional Interfaces
- 🎨 Decorator → Function Composition
- 📋 Template Method → Higher-Order Functions

**04** **Genel Değerlendirme**
Ne zaman OOP, ne zaman FP?

⏱️ 25 dakika • Java 8–26 odaklı

# Java 8 → Java 26

Fonksiyonel programlama yolculuğu – özet harita

FP altyapısı · Veri modelleme · Tip & pattern matching · Son sürümler

# Java 8 → Java 26

Fonksiyonel programlama yolculuğu – özet harita

● FP altyapısı ● Veri modelleme ● Tip & pattern matching ● Son sürümler

**2014 – KIRILMA NOKTASI**

Java 8 · 2014

### Lambda + Functional Interfaces

Davranış birinci sınıf; `Consumer`, `Function`, `Predicate`.

FP kapısı açıldı

Java 8 · 2014

### Stream API

`filter` → `map` → `reduce`; lazy boru hatları.

Iterator → Stream

Java 8 · 2014

### Optional + Default Methods

Null güvenliği; arayüzlere davranış – API'yi kırmadan evrim.

API evrimi

# Java 8 → Java 26

Fonksiyonel programlama yolculuğu – özet harita

FP altyapısı · Veri modelleme · Tip & pattern matching · Son sürümler

## 2014 – KIRILMA NOKTASI

Java 8 · 2014

### Lambda + Functional Interfaces

Davranış birinci sınıf; `Consumer`, `Function`, `Predicate`.

FP kapısı açıldı

Java 8 · 2014

### Stream API

`filter` → `map` → `reduce`; lazy boru hatları.

Iterator → Stream

Java 8 · 2014

### Optional + Default Methods

Null güvenliği; arayüzlere davranış – API'yi kırmadan evrim.

API evrimi

## 2018-2023 – VERI & TIP SISTEMI

Records · 14→16

### Records

Değişmez taşıyıcılar; `equals/hashCode/toString`.

Veri ≠ davranış

Sealed · 17 LTS

### Sealed Classes

Kontrollü hiyerarşi; derleyici tüm alt tipleri bilir.

ADT

Java 21 LTS

### Switch + record patterns

Eksiksiz dallanma; okunabilir `switch`.

Pattern matching

# Java 8 → Java 26

Fonksiyonel programlama yolculuğu – özet harita

FP altyapısı · Veri modelleme · Tip & pattern matching · Son sürümler

## 2014 – KIRILMA NOKTASI

Java 8 · 2014

### Lambda + Functional Interfaces

Davranış birinci sınıf; `Consumer`, `Function`, `Predicate`.

FP kapısı açıldı

Java 8 · 2014

### Stream API

`filter` → `map` → `reduce`; lazy boru hatları.

Iterator → Stream

Java 8 · 2014

### Optional + Default Methods

Null güvenliği; arayüzlere davranış – API'yi kırmadan evrim.

API evrimi

## 2018-2023 – VERI & TIP SISTEMI

Records · 14→16

### Records

Değişmez taşıyıcılar; `equals/hashCode/toString`.

Veri ≠ davranış

Sealed · 17 LTS

### Sealed Classes

Kontrollü hiyerarşi; derleyici tüm alt tipleri bilir.

ADT

Java 21 LTS

### Switch + record patterns

Eksiksiz dallanma; okunabilir `switch`.

Pattern matching

## JAVA 24-26 · FP ILE ÖNE ÇIKANLAR

JEP 485 · Java 24

### Stream Gatherers

Stream'e özelleştirilebilir ara `gather` adımları – `map/filter` ötesi FP tarzı boru hatları.

Stream + kompozisyon

JEP 500 · Java 26

### Make final mean final

Final alanlar reflection'a karşı gerçekten değişmez.

# Design Patterns

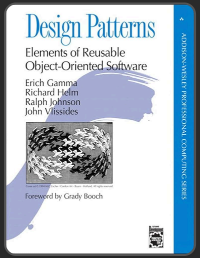
# Design Patterns

## Amaç

Tekrarlanan yazılım tasarım problemlerine **kanıtlanmış çözümler** sunar. Kod kalitesini, bakımını ve yeniden kullanılabilirliğini artırır.

**Gang of Four (GoF)**: Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides

Kitap: **Design Patterns: Elements of Reusable Object-Oriented Software** (1994)



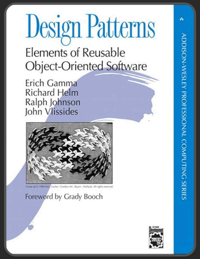
# Design Patterns

## Amaç

Tekrarlanan yazılım tasarım problemlerine **kanıtlanmış çözümler** sunar. Kod kalitesini, bakımını ve yeniden kullanılabilirliğini artırır.

**Gang of Four (GoF)**: Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides

Kitap: **Design Patterns: Elements of Reusable Object-Oriented Software** (1994)




## Genel Bilgiler

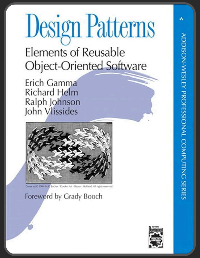
# Design Patterns

## Amaç

Tekrarlanan yazılım tasarım problemlerine **kanıtlanmış çözümler** sunar. Kod kalitesini, bakımını ve yeniden kullanılabilirliğini artırır.

**Gang of Four (GoF)**: Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides

Kitap: **Design Patterns: Elements of Reusable Object-Oriented Software** (1994)




## Genel Bilgiler

**Creational:** Nesne oluşturma sorumluluğunu yönetir (Builder, Factory...)

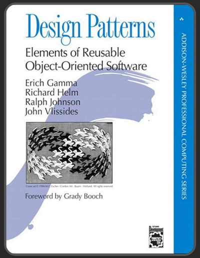
# Design Patterns

## Amaç

Tekrarlanan yazılım tasarım problemlerine **kanıtlanmış çözümler** sunar. Kod kalitesini, bakımını ve yeniden kullanılabilirliğini artırır.

**Gang of Four (GoF)**: Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides

Kitap: **Design Patterns: Elements of Reusable Object-Oriented Software** (1994)




## Genel Bilgiler

**Creational:** Nesne oluşturma sorumluluğunu yönetir (Builder, Factory...)

**Structural:** Sınıf ve nesneleri bir araya getirip esnek bir yapı kurar (Decorator, Adapter...)

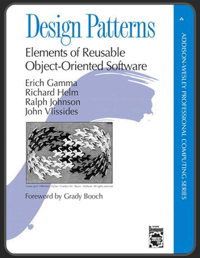
# Design Patterns

## Amaç

Tekrarlanan yazılım tasarım problemlerine **kanıtlanmış çözümler** sunar. Kod kalitesini, bakımını ve yeniden kullanılabilirliğini artırır.

**Gang of Four (GoF)**: Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides

Kitap: **Design Patterns: Elements of Reusable Object-Oriented Software** (1994)




## Genel Bilgiler

**Creational:** Nesne oluşturma sorumluluğunu yönetir (Builder, Factory...)

**Structural:** Sınıf ve nesneleri bir araya getirip esnek bir yapı kurar (Decorator, Adapter...)

**Behavioral:** Nesneler arası iletişim ve davranış akışını düzenler (Strategy, State...)

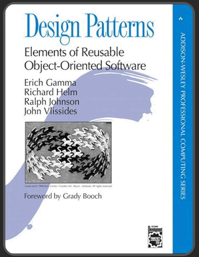
# Design Patterns

## Amaç

Tekrarlanan yazılım tasarım problemlerine **kanıtlanmış çözümler** sunar. Kod kalitesini, bakımını ve yeniden kullanılabilirliğini artırır.

**Gang of Four (GoF)**: Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides

Kitap: **Design Patterns: Elements of Reusable Object-Oriented Software** (1994)




## Genel Bilgiler

**Creational:** Nesne oluşturma sorumluluğunu yönetir (Builder, Factory...)

**Structural:** Sınıf ve nesneleri bir araya getirip esnek bir yapı kurar (Decorator, Adapter...)

**Behavioral:** Nesneler arası iletişim ve davranış akışını düzenler (Strategy, State...)

### Modern Java ile Değişim

Pattern'ler aynı kalır; uygulama şekli modernleşir. Lambda, Stream API, records, sealed classes ve pattern matching ile daha az kod, daha net okunabilirlik ve daha kolay test elde edilir.

# Sunumdaki Pattern’ler

# Sunumdaki Pattern'ler

**Builder**

Immutability
Records

# Sunumdaki Pattern'ler

**Builder**

Immutability
Records

**Strategy**

Lambda Expressions
Functional Interfaces

# Sunumdaki Pattern'ler

**Builder**
Immutability
Records

**Strategy**
Lambda Expressions
Functional Interfaces

**Decorator**
Function Composition
andThen()

# Sunumdaki Pattern'ler

🏗️

**Builder**

Immutability
Records

🎯

**Strategy**

Lambda Expressions
Functional Interfaces

🎨

**Decorator**

Function Composition
andThen()

📋

**Template Method**

Higher-Order
Functions

# Sunumdaki Pattern'ler

🏗️

**Builder**

Immutability
Records

🎯

**Strategy**

Lambda Expressions
Functional Interfaces

🎨

**Decorator**

Function Composition
andThen()

📋

**Template Method**

Higher-Order
Functions

Her pattern için **solda klasik OOP** ve **sağda modern FP** yaklaşımını yan yana göreceğiz

# 🏗️ Builder Pattern – klasik OOP

**Tanım:** Karmaşık nesneyi *adım adım* kurup `build()` ile tek seferde doğrulayan creational pattern. Telescoping constructor yerine okunabilir fluent API.

**Telescoping**

```
new Thing("a","b", null, null, BigDecimal.ZERO);
```

**Mutable ara durum**

Builder'da fluent setter'lar ( `requestedQuantity` vb.) ara alanları değiştirir.

**İki katman**

Ürün immutable olsa da *kurulum süreci* mutasyon içerir.

**Uzun build()**

Çok alanda doğrulama tek metotta şişebilir.

# 🏗️ Builder Pattern — klasik OOP

**Tanım:** Karmaşık nesneyi *adım adım* kurup `build()` ile tek seferde doğrulayan creational pattern. Telescoping constructor yerine okunabilir fluent API.

**Telescoping**

```
new Thing("a","b", null, null, BigDecimal.ZERO);
```


**Mutable ara durum**

Builder'da fluent setter'lar ( `requestedQuantity` vb.) ara alanları değiştirir.

**İki katman**

Ürün immutable olsa da *kurulum süreci* mutasyon içerir.

**Uzun build()**

Çok alanda doğrulama tek metotta şişebilir.

Klasik OOP — `StockReservationClassic`

Modern — record kompozisyonu

Klasik OOP — `StockReservationClassic`

Modern — record kompozisyonu

Klasik OOP — `StockReservationClassic`

```java
// classic/builder/StockReservationClassic.java — yapı özeti
public final class StockReservationClassic {

    // private final alanlar (~20)

    private StockReservationClassic(Builder builder) {
        // builder alanlarından atama
    }

    // getter'lar …
    @Override public String toString() { /* … */ }
    @Override public boolean equals(Object o) { /* reservationId */ }
    @Override public int hashCode() { return Objects.hash(reservationId); }

    public static class Builder {
        private final String reservationId;
        private final String warehouseId;
        private final String sku;
        private int requestedQuantity;
        // reservedQuantity, unitCost, discountPercent, vatPercent, currencyCode, aisl
        // grossWeightKg, netWeightKg, bestBeforeDate, batchNumber, lotSerial, hazmat(

        public Builder(String reservationId, String warehouseId, String sku) {
            this.reservationId = reservationId;
            this.warehouseId = warehouseId;
            this.sku = sku;
        }

        public Builder requestedQuantity(int requestedQuantity) {
            this.requestedQuantity = requestedQuantity;
            return this;
        }
        // … fluent setter'lar

        public StockReservationClassic build() {
            // validasyonlar — çoğu kural bu metotta
            if (reservationId == null || reservationId.isBlank())
                throw new IllegalStateException("Rezervasyon ID zorunludur");
            // …
            return new StockReservationClassic(this);
        }
    }
}
```

Modern — record kompozisyonu

## Klasik OOP — `StockReservationClassic`

```java
// classic/builder/StockReservationClassic.java — yapı özeti
public final class StockReservationClassic {

    // private final alanlar (~20)

    private StockReservationClassic(Builder builder) {
        // builder alanlarından atama
    }

    // getter'lar …
    @Override public String toString() { /* … */ }
    @Override public boolean equals(Object o) { /* reservationId */ }
    @Override public int hashCode() { return Objects.hash(reservationId); }

    public static class Builder {
        private final String reservationId;
        private final String warehouseId;
        private final String sku;
        private int requestedQuantity;
        // reservedQuantity, unitCost, discountPercent, vatPercent, currencyCode, aisl
        // grossWeightKg, netWeightKg, bestBeforeDate, batchNumber, lotSerial, hazmat(

        public Builder(String reservationId, String warehouseId, String sku) {
            this.reservationId = reservationId;
            this.warehouseId = warehouseId;
            this.sku = sku;
        }

        public Builder requestedQuantity(int requestedQuantity) {
            this.requestedQuantity = requestedQuantity;
            return this;
        }
        // … fluent setter'lar

        public StockReservationClassic build() {
            // validasyonlar — çoğu kural bu metotta
            if (reservationId == null || reservationId.isBlank())
                throw new IllegalStateException("Rezervasyon ID zorunludur");
            // …
            return new StockReservationClassic(this);
        }
    }
}
```

## Modern — record kompozisyonu

```java
// modern/builder/StockReservation.java — aggregate
public record StockReservation(
    ReservationIdentity identity,
    QuantityAllocation quantities,
    PricingTerms pricing,
    StorageSlot storageSlot,
    PhysicalWeights physicalWeights,
    LotTraceability traceability,
    ComplianceNotes compliance
) {}
```

## Klasik OOP — `StockReservationClassic`

```java
// classic/builder/StockReservationClassic.java — yapı özeti
public final class StockReservationClassic {

    // private final alanlar (~20)

    private StockReservationClassic(Builder builder) {
        // builder alanlarından atama
    }

    // getter'lar …
    @Override public String toString() { /* … */ }
    @Override public boolean equals(Object o) { /* reservationId */ }
    @Override public int hashCode() { return Objects.hash(reservationId); }

    public static class Builder {
        private final String reservationId;
        private final String warehouseId;
        private final String sku;
        private int requestedQuantity;
        // reservedQuantity, unitCost, discountPercent, vatPercent, currencyCode, aisl
        // grossWeightKg, netWeightKg, bestBeforeDate, batchNumber, lotSerial, hazmat(

        public Builder(String reservationId, String warehouseId, String sku) {
            this.reservationId = reservationId;
            this.warehouseId = warehouseId;
            this.sku = sku;
        }

        public Builder requestedQuantity(int requestedQuantity) {
            this.requestedQuantity = requestedQuantity;
            return this;
        }
        // … fluent setter'lar

        public StockReservationClassic build() {
            // validasyonlar — çoğu kural bu metotta
            if (reservationId == null || reservationId.isBlank())
                throw new IllegalStateException("Rezervasyon ID zorunludur");
            // …
            return new StockReservationClassic(this);
        }
    }
}
```

## Modern — record kompozisyonu

```java
// modern/builder/StockReservation.java — aggregate
public record StockReservation(
    ReservationIdentity identity,
    QuantityAllocation quantities,
    PricingTerms pricing,
    StorageSlot storageSlot,
    PhysicalWeights physicalWeights,
    LotTraceability traceability,
    ComplianceNotes compliance
) {}
```

```java
// Alt tiplerde validasyon (compact constructor) — örnek: ReservationIdentity
public record ReservationIdentity(String reservationId, String warehouseId, String sku
    public ReservationIdentity {
        if (reservationId == null || reservationId.isBlank())
            throw new IllegalArgumentException("Rezervasyon ID zorunludur");
        // warehouseId, sku …
    }
}
```

## Klasik OOP — `StockReservationClassic`

```java
 // classic/builder/StockReservationClassic.java — yapı özeti
public final class StockReservationClassic {

    // private final alanlar (~20)

    private StockReservationClassic(Builder builder) {
        // builder alanlarından atama
    }

    // getter'lar …
    @Override public String toString() { /* … */ }
    @Override public boolean equals(Object o) { /* reservationId */ }
    @Override public int hashCode() { return Objects.hash(reservationId); }

    public static class Builder {
        private final String reservationId;
        private final String warehouseId;
        private final String sku;
        private int requestedQuantity;
        // reservedQuantity, unitCost, discountPercent, vatPercent, currencyCode, aisl
        // grossWeightKg, netWeightKg, bestBeforeDate, batchNumber, lotSerial, hazmat(

        public Builder(String reservationId, String warehouseId, String sku) {
            this.reservationId = reservationId;
            this.warehouseId = warehouseId;
            this.sku = sku;
        }

        public Builder requestedQuantity(int requestedQuantity) {
            this.requestedQuantity = requestedQuantity;
            return this;
        }
        // … fluent setter'lar

        public StockReservationClassic build() {
            // validasyonlar — çoğu kural bu metotta
            if (reservationId == null || reservationId.isBlank())
                throw new IllegalStateException("Rezervasyon ID zorunludur");
            // …
            return new StockReservationClassic(this);
        }
    }
}
```

## Modern — record kompozisyonu

```java
 // modern/builder/StockReservation.java — aggregate
public record StockReservation(
    ReservationIdentity identity,
    QuantityAllocation quantities,
    PricingTerms pricing,
    StorageSlot storageSlot,
    PhysicalWeights physicalWeights,
    LotTraceability traceability,
    ComplianceNotes compliance
) {}
```

```java
 // Alt tiplerde validasyon (compact constructor) — örnek: ReservationIdentity
public record ReservationIdentity(String reservationId, String warehouseId, String sku
    public ReservationIdentity {
        if (reservationId == null || reservationId.isBlank())
            throw new IllegalArgumentException("Rezervasyon ID zorunludur");
        // warehouseId, sku …
    }
}
```

```java
 // Kullanım: factory / defaults ile birleştirme
var res = new StockReservation(
    new ReservationIdentity("R-1", "WH-1", "SKU-A"),
    new QuantityAllocation(100, 80),
    new PricingTerms(new BigDecimal("10"), BigDecimal.ZERO, BigDecimal.ZERO, "TRY"),
    StorageSlot.defaults(),
    PhysicalWeights.none(),
    LotTraceability.empty(),
    ComplianceNotes.empty()
);
```

Özet: doğrulama `build()` yığınında değil; küçük record'ların ctor'unda. Aggregate yalnızca kompozisyon.

## Klasik OOP — `StockReservationClassic`

```java
// classic/builder/StockReservationClassic.java — yapı özeti
public final class StockReservationClassic {

    // private final alanlar (~20)

    private StockReservationClassic(Builder builder) {
        // builder alanlarından atama
    }

    // getter'lar …
    @Override public String toString() { /* … */ }
    @Override public boolean equals(Object o) { /* reservationId */ }
    @Override public int hashCode() { return Objects.hash(reservationId); }

    public static class Builder {
        private final String reservationId;
        private final String warehouseId;
        private final String sku;
        private int requestedQuantity;
        // reservedQuantity, unitCost, discountPercent, vatPercent, currencyCode, aisl
        // grossWeightKg, netWeightKg, bestBeforeDate, batchNumber, lotSerial, hazmat(

        public Builder(String reservationId, String warehouseId, String sku) {
            this.reservationId = reservationId;
            this.warehouseId = warehouseId;
            this.sku = sku;
        }

        public Builder requestedQuantity(int requestedQuantity) {
            this.requestedQuantity = requestedQuantity;
            return this;
        }
        // … fluent setter'lar

        public StockReservationClassic build() {
            // validasyonlar — çoğu kural bu metotta
            if (reservationId == null || reservationId.isBlank())
                throw new IllegalStateException("Rezervasyon ID zorunludur");
            // …
            return new StockReservationClassic(this);
        }
    }
}
```

## Modern — record kompozisyonu

```java
// modern/builder/StockReservation.java — aggregate
public record StockReservation(
    ReservationIdentity identity,
    QuantityAllocation quantities,
    PricingTerms pricing,
    StorageSlot storageSlot,
    PhysicalWeights physicalWeights,
    LotTraceability traceability,
    ComplianceNotes compliance
) {}
```

```java
// Alt tiplerde validasyon (compact constructor) — örnek: ReservationIdentity
public record ReservationIdentity(String reservationId, String warehouseId, String sku
    public ReservationIdentity {
        if (reservationId == null || reservationId.isBlank())
            throw new IllegalArgumentException("Rezervasyon ID zorunludur");
        // warehouseId, sku …
    }
}
```

```java
// Kullanım: factory / defaults ile birleştirme
var res = new StockReservation(
    new ReservationIdentity("R-1", "WH-1", "SKU-A"),
    new QuantityAllocation(100, 80),
    new PricingTerms(new BigDecimal("10"), BigDecimal.ZERO, BigDecimal.ZERO, "TRY"),
    StorageSlot.defaults(),
    PhysicalWeights.none(),
    LotTraceability.empty(),
    ComplianceNotes.empty()
);
```

Özet: doğrulama `build()` yığınında değil; küçük record'ların ctor'unda. Aggregate yalnızca kompozisyon.

**Immutability:** alanlar sabit; eksik opsiyoneller `defaults()` / `empty()` ile doldurulur.

## Klasik OOP — `StockReservationClassic`

```java
// classic/builder/StockReservationClassic.java — yapı özeti
public final class StockReservationClassic {

    // private final alanlar (~20)

    private StockReservationClassic(Builder builder) {
        // builder alanlarından atama
    }

    // getter'lar …
    @Override public String toString() { /* … */ }
    @Override public boolean equals(Object o) { /* reservationId */ }
    @Override public int hashCode() { return Objects.hash(reservationId); }

    public static class Builder {
        private final String reservationId;
        private final String warehouseId;
        private final String sku;
        private int requestedQuantity;
        // reservedQuantity, unitCost, discountPercent, vatPercent, currencyCode, aisl
        // grossWeightKg, netWeightKg, bestBeforeDate, batchNumber, lotSerial, hazmat(

        public Builder(String reservationId, String warehouseId, String sku) {
            this.reservationId = reservationId;
            this.warehouseId = warehouseId;
            this.sku = sku;
        }

        public Builder requestedQuantity(int requestedQuantity) {
            this.requestedQuantity = requestedQuantity;
            return this;
        }
        // … fluent setter'lar

        public StockReservationClassic build() {
            // validasyonlar — çoğu kural bu metotta
            if (reservationId == null || reservationId.isBlank())
                throw new IllegalStateException("Rezervasyon ID zorunludur");
            // …
            return new StockReservationClassic(this);
        }
    }
}
```

## Modern — record kompozisyonu

```java
// modern/builder/StockReservation.java — aggregate
public record StockReservation(
    ReservationIdentity identity,
    QuantityAllocation quantities,
    PricingTerms pricing,
    StorageSlot storageSlot,
    PhysicalWeights physicalWeights,
    LotTraceability traceability,
    ComplianceNotes compliance
) {}
```

```java
// Alt tiplerde validasyon (compact constructor) — örnek: ReservationIdentity
public record ReservationIdentity(String reservationId, String warehouseId, String sku
    public ReservationIdentity {
        if (reservationId == null || reservationId.isBlank())
            throw new IllegalArgumentException("Rezervasyon ID zorunludur");
        // warehouseId, sku …
    }
}
```

```java
// Kullanım: factory / defaults ile birleştirme
var res = new StockReservation(
    new ReservationIdentity("R-1", "WH-1", "SKU-A"),
    new QuantityAllocation(100, 80),
    new PricingTerms(new BigDecimal("10"), BigDecimal.ZERO, BigDecimal.ZERO, "TRY"),
    StorageSlot.defaults(),
    PhysicalWeights.none(),
    LotTraceability.empty(),
    ComplianceNotes.empty()
);
```

Özet: doğrulama `build()` yığınında değil; küçük record'ların ctor'unda. Aggregate yalnızca kompozisyon.

**Immutability:** alanlar sabit; eksik opsiyoneller `defaults()` / `empty()` ile doldurulur.

**Parçalı kurallar:** fiyat, miktar, uyumluluk ayrı dosyalarda test edilebilir; tek dev `build()` bloğu şişmez.

# 🎯 Strategy Pattern

**Tanım:** Strategy Pattern, bir algoritma ailesini ayrı sınıflar/fonksiyonlar olarak kapsüller ve çalışma anında uygun olanı seçmeyi sağlar.

🎯 Klasik OOP – Strategy

Modern FP – Strategy

🎯 Klasik OOP – Strategy

Modern FP – Strategy

## 🎯 Klasik OOP – Strategy

```java
 // classic/strategy/PaymentStrategy.java
public interface PaymentStrategy {
    void pay(BigDecimal amount);
}
```

## Modern FP – Strategy

## 🎯 Klasik OOP – Strategy

```java
// classic/strategy/PaymentStrategy.java
public interface PaymentStrategy {
    void pay(BigDecimal amount);
}
```

```java
// classic/strategy/PaymentService.java
public class PaymentService {
    private PaymentStrategy strategy;
    public PaymentService(PaymentStrategy strategy) { this.strategy = strateg
    public void processPayment(BigDecimal amount) { strategy.pay(amount); }
}
```

## Modern FP – Strategy

## 🎯 Klasik OOP – Strategy

```java
// classic/strategy/PaymentStrategy.java
public interface PaymentStrategy {
    void pay(BigDecimal amount);
}
```

```java
// classic/strategy/PaymentService.java
public class PaymentService {
    private PaymentStrategy strategy;
    public PaymentService(PaymentStrategy strategy) { this.strategy = strateg
    public void processPayment(BigDecimal amount) { strategy.pay(amount); }
}
```

```java
// classic/strategy/CreditCardPayment.java
public class CreditCardPayment implements PaymentStrategy {
    @Override
    public void pay(BigDecimal amount) { IO.println("Kredi karti ile odeme: "
}
```

## Modern FP – Strategy

## 🎯 Klasik OOP – Strategy

```java
// classic/strategy/PaymentStrategy.java
public interface PaymentStrategy {
    void pay(BigDecimal amount);
}
```

```java
// classic/strategy/PaymentService.java
public class PaymentService {
    private PaymentStrategy strategy;
    public PaymentService(PaymentStrategy strategy) { this.strategy = strateg
    public void processPayment(BigDecimal amount) { strategy.pay(amount); }
}
```

```java
// classic/strategy/CreditCardPayment.java
public class CreditCardPayment implements PaymentStrategy {
    @Override
    public void pay(BigDecimal amount) { IO.println("Kredi karti ile odeme: "
}
```

```java
// classic/strategy/StrategyDemo.java
BigDecimal amount = new BigDecimal("299.99");
PaymentService service = new PaymentService(new CreditCardPayment());
service.processPayment(amount);
```

## Modern FP – Strategy

## 🎯 Klasik OOP – Strategy

```java
// classic/strategy/PaymentStrategy.java
public interface PaymentStrategy {
    void pay(BigDecimal amount);
}
```

```java
// classic/strategy/PaymentService.java
public class PaymentService {
    private PaymentStrategy strategy;
    public PaymentService(PaymentStrategy strategy) { this.strategy = strateg
    public void processPayment(BigDecimal amount) { strategy.pay(amount); }
}
```

```java
// classic/strategy/CreditCardPayment.java
public class CreditCardPayment implements PaymentStrategy {
    @Override
    public void pay(BigDecimal amount) { IO.println("Kredi karti ile odeme: "
}
```

```java
// classic/strategy/StrategyDemo.java
BigDecimal amount = new BigDecimal("299.99");
PaymentService service = new PaymentService(new CreditCardPayment());
service.processPayment(amount);
```

## Modern FP – Strategy

```java
// java.util.function paketinde
@FunctionalInterface
public interface Consumer<T> {
    void accept(T t);
}
```

## 🎯 Klasik OOP – Strategy

```java
// classic/strategy/PaymentStrategy.java
public interface PaymentStrategy {
    void pay(BigDecimal amount);
}
```

```java
// classic/strategy/PaymentService.java
public class PaymentService {
    private PaymentStrategy strategy;
    public PaymentService(PaymentStrategy strategy) { this.strategy = strateg
    public void processPayment(BigDecimal amount) { strategy.pay(amount); }
}
```

```java
// classic/strategy/CreditCardPayment.java
public class CreditCardPayment implements PaymentStrategy {
    @Override
    public void pay(BigDecimal amount) { IO.println("Kredi karti ile odeme: "
}
```

```java
// classic/strategy/StrategyDemo.java
BigDecimal amount = new BigDecimal("299.99");
PaymentService service = new PaymentService(new CreditCardPayment());
service.processPayment(amount);
```

## Modern FP – Strategy

```java
// java.util.function paketinde
@FunctionalInterface
public interface Consumer<T> {
    void accept(T t);
}
```

```java
// modern/strategy/PaymentService.java
Consumer<BigDecimal> creditCard = amount ->
    IO.println("Kredi karti ile odeme: " + amount + " TL");
```

## 🎯 Klasik OOP – Strategy

```java
// classic/strategy/PaymentStrategy.java
public interface PaymentStrategy {
    void pay(BigDecimal amount);
}
```

```java
// classic/strategy/PaymentService.java
public class PaymentService {
    private PaymentStrategy strategy;
    public PaymentService(PaymentStrategy strategy) { this.strategy = strateg
    public void processPayment(BigDecimal amount) { strategy.pay(amount); }
}
```

```java
// classic/strategy/CreditCardPayment.java
public class CreditCardPayment implements PaymentStrategy {
    @Override
    public void pay(BigDecimal amount) { IO.println("Kredi karti ile odeme: "
}
```

```java
// classic/strategy/StrategyDemo.java
BigDecimal amount = new BigDecimal("299.99");
PaymentService service = new PaymentService(new CreditCardPayment());
service.processPayment(amount);
```

## Modern FP – Strategy

```java
// java.util.function paketinde
@FunctionalInterface
public interface Consumer<T> {
    void accept(T t);
}
```

```java
// modern/strategy/PaymentService.java
Consumer<BigDecimal> creditCard = amount ->
    IO.println("Kredi karti ile odeme: " + amount + " TL");
```

```java
// modern/strategy/PaymentService.java
public static Consumer<BigDecimal> creditCard(String cardNumber, String cardH
    return amount -> {
        String masked = "****-****-****-" + cardNumber.substring(cardNumber.l
        IO.println("  Kredi karti ile odeme yapildi: " + amount + " TL");
        IO.println("    Kart: " + masked + " | Sahibi: " + cardHolderName);
    };
}
public static void processPayment(Consumer<BigDecimal> strategy, BigDecimal a
    strategy.accept(amount);
}
```

## 🎯 Klasik OOP – Strategy

```java
// classic/strategy/PaymentStrategy.java
public interface PaymentStrategy {
    void pay(BigDecimal amount);
}
```

```java
// classic/strategy/PaymentService.java
public class PaymentService {
    private PaymentStrategy strategy;
    public PaymentService(PaymentStrategy strategy) { this.strategy = strateg
    public void processPayment(BigDecimal amount) { strategy.pay(amount); }
}
```

```java
// classic/strategy/CreditCardPayment.java
public class CreditCardPayment implements PaymentStrategy {
    @Override
    public void pay(BigDecimal amount) { IO.println("Kredi karti ile odeme: "
}
```

```java
// classic/strategy/StrategyDemo.java
BigDecimal amount = new BigDecimal("299.99");
PaymentService service = new PaymentService(new CreditCardPayment());
service.processPayment(amount);
```

**Not**

Strategy'nin ana fikri davranisi nesneden ayirmaktir. Klasikte interface + class, modernde functional interface + lambda ile ayni hedefe ulasiriz.

## Modern FP – Strategy

```java
// java.util.function paketinde
@FunctionalInterface
public interface Consumer<T> {
    void accept(T t);
}
```

```java
// modern/strategy/PaymentService.java
Consumer<BigDecimal> creditCard = amount ->
    IO.println("Kredi karti ile odeme: " + amount + " TL");
```

```java
// modern/strategy/PaymentService.java
public static Consumer<BigDecimal> creditCard(String cardNumber, String cardH
    return amount -> {
        String masked = "****-****-****-" + cardNumber.substring(cardNumber.l
        IO.println("  Kredi karti ile odeme yapildi: " + amount + " TL");
        IO.println("    Kart: " + masked + " | Sahibi: " + cardHolderName);
    };
}
public static void processPayment(Consumer<BigDecimal> strategy, BigDecimal a
    strategy.accept(amount);
}
```

```java
// StrategyDemo.modernApproach(...)
Consumer<BigDecimal> cc = PaymentService.creditCard("...0366", "Ahmet Yilmaz"
PaymentService.processPayment(cc, amount);
```

## 🎯 Klasik OOP – Strategy

```java
// classic/strategy/PaymentStrategy.java
public interface PaymentStrategy {
    void pay(BigDecimal amount);
}
```

```java
// classic/strategy/PaymentService.java
public class PaymentService {
    private PaymentStrategy strategy;
    public PaymentService(PaymentStrategy strategy) { this.strategy = strateg
    public void processPayment(BigDecimal amount) { strategy.pay(amount); }
}
```

```java
// classic/strategy/CreditCardPayment.java
public class CreditCardPayment implements PaymentStrategy {
    @Override
    public void pay(BigDecimal amount) { IO.println("Kredi karti ile odeme: "
}
```

```java
// classic/strategy/StrategyDemo.java
BigDecimal amount = new BigDecimal("299.99");
PaymentService service = new PaymentService(new CreditCardPayment());
service.processPayment(amount);
```

**Not**

Strategy'nin ana fikri davranisi nesneden ayirmaktir. Klasikte interface + class, modernde functional interface + lambda ile ayni hedefe ulasiriz.

**Functional Interface:** Strategy davranisini sinif yerine `Consumer<BigDecimal>` ile temsil ediyoruz.

## Modern FP – Strategy

```java
// java.util.function paketinde
@FunctionalInterface
public interface Consumer<T> {
    void accept(T t);
}
```

```java
// modern/strategy/PaymentService.java
Consumer<BigDecimal> creditCard = amount ->
    IO.println("Kredi karti ile odeme: " + amount + " TL");
```

```java
// modern/strategy/PaymentService.java
public static Consumer<BigDecimal> creditCard(String cardNumber, String cardH
    return amount -> {
        String masked = "****-****-****-" + cardNumber.substring(cardNumber.l
        IO.println("  Kredi karti ile odeme yapildi: " + amount + " TL");
        IO.println("    Kart: " + masked + " | Sahibi: " + cardHolderName);
    };
}
public static void processPayment(Consumer<BigDecimal> strategy, BigDecimal a
    strategy.accept(amount);
}
```

```java
// StrategyDemo.modernApproach(...)
Consumer<BigDecimal> cc = PaymentService.creditCard("...0366", "Ahmet Yilmaz"
PaymentService.processPayment(cc, amount);
```

```java
// runtime'da farkli strateji secimi
Consumer<BigDecimal> bank = PaymentService.bankTransfer("TR...1326", "Garanti
PaymentService.processPayment(bank, amount);
```

## 🎯 Klasik OOP – Strategy

```java
// classic/strategy/PaymentStrategy.java
public interface PaymentStrategy {
    void pay(BigDecimal amount);
}
```

```java
// classic/strategy/PaymentService.java
public class PaymentService {
    private PaymentStrategy strategy;
    public PaymentService(PaymentStrategy strategy) { this.strategy = strateg
    public void processPayment(BigDecimal amount) { strategy.pay(amount); }
}
```

```java
// classic/strategy/CreditCardPayment.java
public class CreditCardPayment implements PaymentStrategy {
    @Override
    public void pay(BigDecimal amount) { IO.println("Kredi karti ile odeme: "
}
```

```java
// classic/strategy/StrategyDemo.java
BigDecimal amount = new BigDecimal("299.99");
PaymentService service = new PaymentService(new CreditCardPayment());
service.processPayment(amount);
```

**Not**

Strategy'nin ana fikri davranisi nesneden ayirmaktir. Klasikte interface + class, modernde functional interface + lambda ile ayni hedefe ulasiriz.

**Functional Interface:** Strategy davranisini sinif yerine `Consumer<BigDecimal>` ile temsil ediyoruz.

**Lambda ve Tip:** Lambda tek basina tip tasimaz; hedef tipi bir functional interface belirler.

## Modern FP – Strategy

```java
// java.util.function paketinde
@FunctionalInterface
public interface Consumer<T> {
    void accept(T t);
}
```

```java
// modern/strategy/PaymentService.java
Consumer<BigDecimal> creditCard = amount ->
    IO.println("Kredi karti ile odeme: " + amount + " TL");
```

```java
// modern/strategy/PaymentService.java
public static Consumer<BigDecimal> creditCard(String cardNumber, String cardH
    return amount -> {
        String masked = "****-****-****-" + cardNumber.substring(cardNumber.l
        IO.println("  Kredi karti ile odeme yapildi: " + amount + " TL");
        IO.println("    Kart: " + masked + " | Sahibi: " + cardHolderName);
    };
}
public static void processPayment(Consumer<BigDecimal> strategy, BigDecimal a
    strategy.accept(amount);
}
```

```java
// StrategyDemo.modernApproach(...)
Consumer<BigDecimal> cc = PaymentService.creditCard("...0366", "Ahmet Yilmaz"
PaymentService.processPayment(cc, amount);
```

```java
// runtime'da farkli strateji secimi
Consumer<BigDecimal> bank = PaymentService.bankTransfer("TR...1326", "Garanti
PaymentService.processPayment(bank, amount);
```

## 🎯 Klasik OOP – Strategy

```java
// classic/strategy/PaymentStrategy.java
public interface PaymentStrategy {
    void pay(BigDecimal amount);
}
```

```java
// classic/strategy/PaymentService.java
public class PaymentService {
    private PaymentStrategy strategy;
    public PaymentService(PaymentStrategy strategy) { this.strategy = strateg
    public void processPayment(BigDecimal amount) { strategy.pay(amount); }
}
```

```java
// classic/strategy/CreditCardPayment.java
public class CreditCardPayment implements PaymentStrategy {
    @Override
    public void pay(BigDecimal amount) { IO.println("Kredi karti ile odeme: "
}
```

```java
// classic/strategy/StrategyDemo.java
BigDecimal amount = new BigDecimal("299.99");
PaymentService service = new PaymentService(new CreditCardPayment());
service.processPayment(amount);
```

**Not**

Strategy'nin ana fikri davranisi nesneden ayirmaktir. Klasikte interface + class, modernde functional interface + lambda ile ayni hedefe ulasiriz.

**Functional Interface:** Strategy davranisini sinif yerine `Consumer<BigDecimal>` ile temsil ediyoruz.

**Lambda ve Tip:** Lambda tek basina tip tasimaz; hedef tipi bir functional interface belirler.

**Daha Az Boilerplate:** Daha az sinif, daha kisa kod, daha kolay bakim ve test.

## Modern FP – Strategy

```java
// java.util.function paketinde
@FunctionalInterface
public interface Consumer<T> {
    void accept(T t);
}
```

```java
// modern/strategy/PaymentService.java
Consumer<BigDecimal> creditCard = amount ->
    IO.println("Kredi karti ile odeme: " + amount + " TL");
```

```java
// modern/strategy/PaymentService.java
public static Consumer<BigDecimal> creditCard(String cardNumber, String cardH
    return amount -> {
        String masked = "****-****-****-" + cardNumber.substring(cardNumber.l
        IO.println("  Kredi karti ile odeme yapildi: " + amount + " TL");
        IO.println("    Kart: " + masked + " | Sahibi: " + cardHolderName);
    };
}
public static void processPayment(Consumer<BigDecimal> strategy, BigDecimal a
    strategy.accept(amount);
}
```

```java
// StrategyDemo.modernApproach(...)
Consumer<BigDecimal> cc = PaymentService.creditCard("...0366", "Ahmet Yilmaz"
PaymentService.processPayment(cc, amount);
```

```java
// runtime'da farkli strateji secimi
Consumer<BigDecimal> bank = PaymentService.bankTransfer("TR...1326", "Garanti
PaymentService.processPayment(bank, amount);
```

## 🎯 Klasik OOP – Strategy

```java
// classic/strategy/PaymentStrategy.java
public interface PaymentStrategy {
    void pay(BigDecimal amount);
}
```

```java
// classic/strategy/PaymentService.java
public class PaymentService {
    private PaymentStrategy strategy;
    public PaymentService(PaymentStrategy strategy) { this.strategy = strateg
    public void processPayment(BigDecimal amount) { strategy.pay(amount); }
}
```

```java
// classic/strategy/CreditCardPayment.java
public class CreditCardPayment implements PaymentStrategy {
    @Override
    public void pay(BigDecimal amount) { IO.println("Kredi karti ile odeme: "
}
```

```java
// classic/strategy/StrategyDemo.java
BigDecimal amount = new BigDecimal("299.99");
PaymentService service = new PaymentService(new CreditCardPayment());
service.processPayment(amount);
```

**Not**

Strategy'nin ana fikri davranisi nesneden ayirmaktir. Klasikte interface + class, modernde functional interface + lambda ile ayni hedefe ulasiriz.

**Functional Interface:**Strategy davranisini sinif yerine `Consumer<BigDecimal>` ile temsil ediyoruz.

**Lambda ve Tip:**Lambda tek basina tip tasimaz; hedef tipi bir functional interface belirler.

**Daha Az Boilerplate:**Daha az sinif, daha kisa kod, daha kolay bakim ve test.

## Modern FP – Strategy

```java
// java.util.function paketinde
@FunctionalInterface
public interface Consumer<T> {
    void accept(T t);
}
```

```java
// modern/strategy/PaymentService.java
Consumer<BigDecimal> creditCard = amount ->
    IO.println("Kredi karti ile odeme: " + amount + " TL");
```

```java
// modern/strategy/PaymentService.java
public static Consumer<BigDecimal> creditCard(String cardNumber, String cardH
    return amount -> {
        String masked = "****-****-****-" + cardNumber.substring(cardNumber.l
        IO.println("  Kredi karti ile odeme yapildi: " + amount + " TL");
        IO.println("    Kart: " + masked + " | Sahibi: " + cardHolderName);
    };
}
public static void processPayment(Consumer<BigDecimal> strategy, BigDecimal a
    strategy.accept(amount);
}
```

```java
// StrategyDemo.modernApproach(...)
Consumer<BigDecimal> cc = PaymentService.creditCard("...0366", "Ahmet Yilmaz"
PaymentService.processPayment(cc, amount);
```

```java
// runtime'da farkli strateji secimi
Consumer<BigDecimal> bank = PaymentService.bankTransfer("TR...1326", "Garanti
PaymentService.processPayment(bank, amount);
```

# 🎨 Decorator Pattern

**Tanım:** Decorator, bir nesnenin davranışını ana sınıfı değiştirmeden katman katman genişletmeyi sağlayan yapısal bir pattern'dir.

Klasik OOP – Decorator

Modern FP – Decorator

Klasik OOP – Decorator

Modern FP – Decorator

## Klasik OOP – Decorator

```java
public interface OrderService {
    Order process(Order order);
}
```

## Modern FP – Decorator

## Klasik OOP – Decorator

```java
public interface OrderService {
    Order process(Order order);
}
```

```java
public class BasicOrderService implements OrderService {
    @Override
    public Order process(Order order) {
        return order; // temel islem; ek ozellik yok
    }
}
```

## Modern FP – Decorator

## Klasik OOP – Decorator

```java
public interface OrderService {
    Order process(Order order);
}
```

```java
public class BasicOrderService implements OrderService {
    @Override
    public Order process(Order order) {
        return order; // temel islem; ek ozellik yok
    }
}
```

```java
public abstract class OrderServiceDecorator implements OrderService {
    protected final OrderService wrapped;
    protected OrderServiceDecorator(OrderService wrapped) {
        this.wrapped = wrapped;
    }
}
```

## Modern FP – Decorator

## Klasik OOP – Decorator

```java
public interface OrderService {
    Order process(Order order);
}
```

```java
public class BasicOrderService implements OrderService {
    @Override
    public Order process(Order order) {
        return order; // temel islem; ek ozellik yok
    }
}
```

```java
public abstract class OrderServiceDecorator implements OrderService {
    protected final OrderService wrapped;
    protected OrderServiceDecorator(OrderService wrapped) {
        this.wrapped = wrapped;
    }
}
```

```java
public class GiftWrapDecorator extends OrderServiceDecorator {
    public GiftWrapDecorator(OrderService wrapped) { super(wrapped); }
    @Override
    public Order process(Order order) {
        Order processed = wrapped.process(order); // once icteki zincir
        return processed.addFeature("Hediye Paketi", new BigDecimal("15"));
    }
}
```

## Modern FP – Decorator

Klasik OOP – Decorator

```java
public interface OrderService {
    Order process(Order order);
}
```

```java
public class BasicOrderService implements OrderService {
    @Override
    public Order process(Order order) {
        return order; // temel islem; ek ozellik yok
    }
}
```

```java
public abstract class OrderServiceDecorator implements OrderService {
    protected final OrderService wrapped;
    protected OrderServiceDecorator(OrderService wrapped) {
        this.wrapped = wrapped;
    }
}
```

```java
public class GiftWrapDecorator extends OrderServiceDecorator {
    public GiftWrapDecorator(OrderService wrapped) { super(wrapped); }
    @Override
    public Order process(Order order) {
        Order processed = wrapped.process(order); // once icteki zincir
        return processed.addFeature("Hediye Paketi", new BigDecimal("15"));
    }
}
```

```java
OrderService service = new ExpressShippingDecorator(
    new InsuranceDecorator(
        new GiftWrapDecorator(new BasicOrderService())
    )
);
```

Modern FP – Decorator

## Klasik OOP – Decorator

```java
public interface OrderService {
    Order process(Order order);
}
```

```java
public class BasicOrderService implements OrderService {
    @Override
    public Order process(Order order) {
        return order; // temel islem; ek ozellik yok
    }
}
```

```java
public abstract class OrderServiceDecorator implements OrderService {
    protected final OrderService wrapped;
    protected OrderServiceDecorator(OrderService wrapped) {
        this.wrapped = wrapped;
    }
}
```

```java
public class GiftWrapDecorator extends OrderServiceDecorator {
    public GiftWrapDecorator(OrderService wrapped) { super(wrapped); }
    @Override
    public Order process(Order order) {
        Order processed = wrapped.process(order); // once icteki zincir
        return processed.addFeature("Hediye Paketi", new BigDecimal("15"));
    }
}
```

```java
OrderService service = new ExpressShippingDecorator(
    new InsuranceDecorator(
        new GiftWrapDecorator(new BasicOrderService())
    )
);
```

## Modern FP – Decorator

```java
// modern/decorator/OrderEnhancer.java
public static UnaryOperator<Order> giftWrap() {
    return order -> order.addFeature("Hediye Paketi", new BigDecimal("15"));
}

public static UnaryOperator<Order> insurance() {
    return order -> order.addFeature("Kargo Sigortasi", new BigDecimal("10"))
}

public static UnaryOperator<Order> expressShipping() {
    return order -> order.addFeature("Hizli Kargo", new BigDecimal("25"));
}
```

## Klasik OOP – Decorator

```java
public interface OrderService {
    Order process(Order order);
}
```

```java
public class BasicOrderService implements OrderService {
    @Override
    public Order process(Order order) {
        return order; // temel islem; ek ozellik yok
    }
}
```

```java
public abstract class OrderServiceDecorator implements OrderService {
    protected final OrderService wrapped;
    protected OrderServiceDecorator(OrderService wrapped) {
        this.wrapped = wrapped;
    }
}
```

```java
public class GiftWrapDecorator extends OrderServiceDecorator {
    public GiftWrapDecorator(OrderService wrapped) { super(wrapped); }
    @Override
    public Order process(Order order) {
        Order processed = wrapped.process(order); // once icteki zincir
        return processed.addFeature("Hediye Paketi", new BigDecimal("15"));
    }
}
```

```java
OrderService service = new ExpressShippingDecorator(
    new InsuranceDecorator(
        new GiftWrapDecorator(new BasicOrderService())
    )
);
```

## Modern FP – Decorator

```java
// modern/decorator/OrderEnhancer.java
public static UnaryOperator<Order> giftWrap() {
    return order -> order.addFeature("Hediye Paketi", new BigDecimal("15"));
}

public static UnaryOperator<Order> insurance() {
    return order -> order.addFeature("Kargo Sigortasi", new BigDecimal("10"))
}

public static UnaryOperator<Order> expressShipping() {
    return order -> order.addFeature("Hizli Kargo", new BigDecimal("25"));
}
```

```java
// Normal kullanim (andThen ile acik zincir):
// Function<Order, Order> flow = OrderEnhancer.giftWrap()
//     .andThen(OrderEnhancer.insurance())
//     .andThen(OrderEnhancer.expressShipping());
```

```java
// Sorunlu kullanim:
// UnaryOperator<Order> campaignFlow = campaign.stream()
//     .reduce(UnaryOperator.identity(), UnaryOperator::andThen);
```

```java
// 2) Reduce ile – dinamik, runtime'da liste degisebilir (calisan surum)
List<UnaryOperator<Order>> campaign = List.of(
    OrderEnhancer.giftWrap(),
    OrderEnhancer.insurance(),
    OrderEnhancer.expressShipping()
);
UnaryOperator<Order> enhanceDynamic = campaign.stream()
    .reduce(UnaryOperator.identity(), (pipeline, step) -> pipeline.andThen(st

var order = new Order("ORD-001", new BigDecimal("200"));
var campaignResult = enhanceDynamic.apply(order);
```

## Klasik OOP – Decorator

```java
public interface OrderService {
    Order process(Order order);
}
```

```java
public class BasicOrderService implements OrderService {
    @Override
    public Order process(Order order) {
        return order; // temel islem; ek ozellik yok
    }
}
```

```java
public abstract class OrderServiceDecorator implements OrderService {
    protected final OrderService wrapped;
    protected OrderServiceDecorator(OrderService wrapped) {
        this.wrapped = wrapped;
    }
}
```

```java
public class GiftWrapDecorator extends OrderServiceDecorator {
    public GiftWrapDecorator(OrderService wrapped) { super(wrapped); }
    @Override
    public Order process(Order order) {
        Order processed = wrapped.process(order); // once icteki zincir
        return processed.addFeature("Hediye Paketi", new BigDecimal("15"));
    }
}
```

```java
OrderService service = new ExpressShippingDecorator(
    new InsuranceDecorator(
        new GiftWrapDecorator(new BasicOrderService())
    )
);
```

**Classic:** Her yeni ozellik icin yeni bir wrapper sinifi gerekir.

## Modern FP – Decorator

```java
// modern/decorator/OrderEnhancer.java
public static UnaryOperator<Order> giftWrap() {
    return order -> order.addFeature("Hediye Paketi", new BigDecimal("15"));
}

public static UnaryOperator<Order> insurance() {
    return order -> order.addFeature("Kargo Sigortasi", new BigDecimal("10"))
}

public static UnaryOperator<Order> expressShipping() {
    return order -> order.addFeature("Hizli Kargo", new BigDecimal("25"));
}
```

```java
// Normal kullanim (andThen ile acik zincir):
// Function<Order, Order> flow = OrderEnhancer.giftWrap()
//     .andThen(OrderEnhancer.insurance())
//     .andThen(OrderEnhancer.expressShipping());
```

```java
// Sorunlu kullanim:
// UnaryOperator<Order> campaignFlow = campaign.stream()
//     .reduce(UnaryOperator.identity(), UnaryOperator::andThen);
```

```java
// 2) Reduce ile — dinamik, runtime'da liste degisebilir (calisan surum)
List<UnaryOperator<Order>> campaign = List.of(
    OrderEnhancer.giftWrap(),
    OrderEnhancer.insurance(),
    OrderEnhancer.expressShipping()
);
UnaryOperator<Order> enhanceDynamic = campaign.stream()
    .reduce(UnaryOperator.identity(), (pipeline, step) -> pipeline.andThen(st

var order = new Order("ORD-001", new BigDecimal("200"));
var campaignResult = enhanceDynamic.apply(order);
```

## Klasik OOP – Decorator

```java
public interface OrderService {
    Order process(Order order);
}
```

```java
public class BasicOrderService implements OrderService {
    @Override
    public Order process(Order order) {
        return order; // temel islem; ek ozellik yok
    }
}
```

```java
public abstract class OrderServiceDecorator implements OrderService {
    protected final OrderService wrapped;
    protected OrderServiceDecorator(OrderService wrapped) {
        this.wrapped = wrapped;
    }
}
```

```java
public class GiftWrapDecorator extends OrderServiceDecorator {
    public GiftWrapDecorator(OrderService wrapped) { super(wrapped); }
    @Override
    public Order process(Order order) {
        Order processed = wrapped.process(order); // once icteki zincir
        return processed.addFeature("Hediye Paketi", new BigDecimal("15"));
    }
}
```

```java
OrderService service = new ExpressShippingDecorator(
    new InsuranceDecorator(
        new GiftWrapDecorator(new BasicOrderService())
    )
);
```

**Classic:** Her yeni ozellik icin yeni bir wrapper sinifi gerekir.

**Modern:** `UnaryOperator` zinciriyle davranislar composable hale gelir.

## Modern FP – Decorator

```java
// modern/decorator/OrderEnhancer.java
public static UnaryOperator<Order> giftWrap() {
    return order -> order.addFeature("Hediye Paketi", new BigDecimal("15"));
}

public static UnaryOperator<Order> insurance() {
    return order -> order.addFeature("Kargo Sigortasi", new BigDecimal("10"))
}

public static UnaryOperator<Order> expressShipping() {
    return order -> order.addFeature("Hizli Kargo", new BigDecimal("25"));
}
```

```java
// Normal kullanim (andThen ile acik zincir):
// Function<Order, Order> flow = OrderEnhancer.giftWrap()
//     .andThen(OrderEnhancer.insurance())
//     .andThen(OrderEnhancer.expressShipping());
```

```java
// Sorunlu kullanim:
// UnaryOperator<Order> campaignFlow = campaign.stream()
//     .reduce(UnaryOperator.identity(), UnaryOperator::andThen);
```

```java
// 2) Reduce ile — dinamik, runtime'da liste degisebilir (calisan surum)
List<UnaryOperator<Order>> campaign = List.of(
    OrderEnhancer.giftWrap(),
    OrderEnhancer.insurance(),
    OrderEnhancer.expressShipping()
);
UnaryOperator<Order> enhanceDynamic = campaign.stream()
    .reduce(UnaryOperator.identity(), (pipeline, step) -> pipeline.andThen(st

var order = new Order("ORD-001", new BigDecimal("200"));
var campaignResult = enhanceDynamic.apply(order);
```

## Klasik OOP – Decorator

```java
public interface OrderService {
    Order process(Order order);
}
```

```java
public class BasicOrderService implements OrderService {
    @Override
    public Order process(Order order) {
        return order; // temel islem; ek ozellik yok
    }
}
```

```java
public abstract class OrderServiceDecorator implements OrderService {
    protected final OrderService wrapped;
    protected OrderServiceDecorator(OrderService wrapped) {
        this.wrapped = wrapped;
    }
}
```

```java
public class GiftWrapDecorator extends OrderServiceDecorator {
    public GiftWrapDecorator(OrderService wrapped) { super(wrapped); }
    @Override
    public Order process(Order order) {
        Order processed = wrapped.process(order); // once icteki zincir
        return processed.addFeature("Hediye Paketi", new BigDecimal("15"));
    }
}
```

```java
OrderService service = new ExpressShippingDecorator(
    new InsuranceDecorator(
        new GiftWrapDecorator(new BasicOrderService())
    )
);
```

**Classic:** Her yeni ozellik icin yeni bir wrapper sinifi gerekir.

**Modern:** `UnaryOperator` zinciriyle davranislar composable hale gelir.

## Modern FP – Decorator

```java
// modern/decorator/OrderEnhancer.java
public static UnaryOperator<Order> giftWrap() {
    return order -> order.addFeature("Hediye Paketi", new BigDecimal("15"));
}

public static UnaryOperator<Order> insurance() {
    return order -> order.addFeature("Kargo Sigortasi", new BigDecimal("10"))
}

public static UnaryOperator<Order> expressShipping() {
    return order -> order.addFeature("Hizli Kargo", new BigDecimal("25"));
}
```

```java
// Normal kullanim (andThen ile acik zincir):
// Function<Order, Order> flow = OrderEnhancer.giftWrap()
//     .andThen(OrderEnhancer.insurance())
//     .andThen(OrderEnhancer.expressShipping());
```

```java
// Sorunlu kullanim:
// UnaryOperator<Order> campaignFlow = campaign.stream()
//     .reduce(UnaryOperator.identity(), UnaryOperator::andThen);
```

```java
// 2) Reduce ile — dinamik, runtime'da liste degisebilir (calisan surum)
List<UnaryOperator<Order>> campaign = List.of(
    OrderEnhancer.giftWrap(),
    OrderEnhancer.insurance(),
    OrderEnhancer.expressShipping()
);
UnaryOperator<Order> enhanceDynamic = campaign.stream()
    .reduce(UnaryOperator.identity(), (pipeline, step) -> pipeline.andThen(st

var order = new Order("ORD-001", new BigDecimal("200"));
var campaignResult = enhanceDynamic.apply(order);
```

**Composition:** `andThen()` soldan saga uygulanir; sira degisince fiyat ve ozellik sirasi da degisir.

**Ortak:** Ana nesne bozulmadan ozellik eklenir; Open/Closed korunur.

# 📋 Template Method Pattern – Rapor Üretimi

**Tanım:** Template Method, bir algoritmanın iskeletini tanımlar; degisen adimlar alt siniflara ya da fonksiyon parametrelerine birakilir.

Ornek: Rapor uretimi – PDF / Excel / HTML / Email. Iskelet ayni, format degisiyor.

Klasik OOP – Template Method

Modern FP – Template Method

Klasik OOP – Template Method

Modern FP – Template Method

## Klasik OOP – Template Method

```java
public abstract class AbstractReportGenerator {
    public final void generate(ReportData data) {
        validateData(data);
        String formatted = formatData(data);
        String output = renderOutput(formatted, data.getTitle());
        sendReport(output);
    }

    protected abstract void validateData(ReportData data);
    protected abstract String formatData(ReportData data);
    protected abstract String renderOutput(String formatted, String title);
    protected abstract void sendReport(String output);
}
```

## Modern FP – Template Method

## Klasik OOP – Template Method

```java
public abstract class AbstractReportGenerator {
    public final void generate(ReportData data) {
        validateData(data);
        String formatted = formatData(data);
        String output = renderOutput(formatted, data.getTitle());
        sendReport(output);
    }

    protected abstract void validateData(ReportData data);
    protected abstract String formatData(ReportData data);
    protected abstract String renderOutput(String formatted, String title);
    protected abstract void sendReport(String output);
}
```

```java
public class PdfReportGenerator extends AbstractReportGenerator {
    @Override
    protected void validateData(ReportData data) { ... }
    @Override
    protected String formatData(ReportData data) { ... }
    @Override
    protected String renderOutput(String formatted, String title) { ... }
    @Override
    protected void sendReport(String output) { ... }
}
// + ExcelReportGenerator   → 4 method override
// + HtmlReportGenerator    → 4 method override
// + EmailReportGenerator   → 4 method override
// = 16 override, 5 dosya
```

## Modern FP – Template Method

## Klasik OOP – Template Method

```java
public abstract class AbstractReportGenerator {
    public final void generate(ReportData data) {
        validateData(data);
        String formatted = formatData(data);
        String output = renderOutput(formatted, data.getTitle());
        sendReport(output);
    }

    protected abstract void validateData(ReportData data);
    protected abstract String formatData(ReportData data);
    protected abstract String renderOutput(String formatted, String title);
    protected abstract void sendReport(String output);
}
```

```java
public class PdfReportGenerator extends AbstractReportGenerator {
    @Override
    protected void validateData(ReportData data) { ... }
    @Override
    protected String formatData(ReportData data) { ... }
    @Override
    protected String renderOutput(String formatted, String title) { ... }
    @Override
    protected void sendReport(String output) { ... }
}
// + ExcelReportGenerator   → 4 method override
// + HtmlReportGenerator    → 4 method override
// + EmailReportGenerator   → 4 method override
// = 16 override, 5 dosya
```

```java
var data = new ReportData("Q1 Satis Raporu", columns, rows);
new PdfReportGenerator().generate(data);
new ExcelReportGenerator().generate(data);
new HtmlReportGenerator().generate(data);
new EmailReportGenerator("yonetim@sirket.com").generate(data);
```

## Modern FP – Template Method

## Klasik OOP – Template Method

```java
public abstract class AbstractReportGenerator {
    public final void generate(ReportData data) {
        validateData(data);
        String formatted = formatData(data);
        String output = renderOutput(formatted, data.getTitle());
        sendReport(output);
    }

    protected abstract void validateData(ReportData data);
    protected abstract String formatData(ReportData data);
    protected abstract String renderOutput(String formatted, String title);
    protected abstract void sendReport(String output);
}
```

```java
public class PdfReportGenerator extends AbstractReportGenerator {
    @Override
    protected void validateData(ReportData data) { … }
    @Override
    protected String formatData(ReportData data) { … }
    @Override
    protected String renderOutput(String formatted, String title) { … }
    @Override
    protected void sendReport(String output) { … }
}
// + ExcelReportGenerator   → 4 method override
// + HtmlReportGenerator    → 4 method override
// + EmailReportGenerator   → 4 method override
// = 16 override, 5 dosya
```

```java
var data = new ReportData("Q1 Satis Raporu", columns, rows);
new PdfReportGenerator().generate(data);
new ExcelReportGenerator().generate(data);
new HtmlReportGenerator().generate(data);
new EmailReportGenerator("yonetim@sirket.com").generate(data);
```

## Modern FP – Template Method

```java
public record ReportGenerator(
    Consumer<ReportData> dataValidator,
    Function<ReportData, String> dataFormatter,
    UnaryOperator<String> outputRenderer,
    Consumer<String> reportSender
) {
    public void generate(ReportData data) {
        dataValidator.accept(data);
        String formatted = dataFormatter.apply(data);
        String output = outputRenderer.apply(formatted);
        reportSender.accept(output);
    }
}
```

## Klasik OOP – Template Method

```java
public abstract class AbstractReportGenerator {
    public final void generate(ReportData data) {
        validateData(data);
        String formatted = formatData(data);
        String output = renderOutput(formatted, data.getTitle());
        sendReport(output);
    }

    protected abstract void validateData(ReportData data);
    protected abstract String formatData(ReportData data);
    protected abstract String renderOutput(String formatted, String title);
    protected abstract void sendReport(String output);
}
```

```java
public class PdfReportGenerator extends AbstractReportGenerator {
    @Override
    protected void validateData(ReportData data) { ... }
    @Override
    protected String formatData(ReportData data) { ... }
    @Override
    protected String renderOutput(String formatted, String title) { ... }
    @Override
    protected void sendReport(String output) { ... }
}
// + ExcelReportGenerator   → 4 method override
// + HtmlReportGenerator    → 4 method override
// + EmailReportGenerator   → 4 method override
// = 16 override, 5 dosya
```

```java
var data = new ReportData("Q1 Satis Raporu", columns, rows);
new PdfReportGenerator().generate(data);
new ExcelReportGenerator().generate(data);
new HtmlReportGenerator().generate(data);
new EmailReportGenerator("yonetim@sirket.com").generate(data);
```

## Modern FP – Template Method

```java
public record ReportGenerator(
    Consumer<ReportData> dataValidator,
    Function<ReportData, String> dataFormatter,
    UnaryOperator<String> outputRenderer,
    Consumer<String> reportSender
) {
    public void generate(ReportData data) {
        dataValidator.accept(data);
        String formatted = dataFormatter.apply(data);
        String output = outputRenderer.apply(formatted);
        reportSender.accept(output);
    }
}
```

```java
public static ReportGenerator pdf() {
    return new ReportGenerator(
        data -> IO.println("  [PDF] Dogrulandi"),
        data -> {
            String headers = String.join(" | ", data.columns());
            String rows = formatRows(data, " | ");
            return headers + "\n" + "-".repeat(40) + "\n" + rows;
        },
        formatted -> "[PDF_DOCUMENT]\n" + formatted,
        output -> IO.println("  [PDF] Yazildi: rapor.pdf")
    );
}
```

## Klasik OOP – Template Method

```java
public abstract class AbstractReportGenerator {
    public final void generate(ReportData data) {
        validateData(data);
        String formatted = formatData(data);
        String output = renderOutput(formatted, data.getTitle());
        sendReport(output);
    }

    protected abstract void validateData(ReportData data);
    protected abstract String formatData(ReportData data);
    protected abstract String renderOutput(String formatted, String title);
    protected abstract void sendReport(String output);
}
```

```java
public class PdfReportGenerator extends AbstractReportGenerator {
    @Override
    protected void validateData(ReportData data) { ... }
    @Override
    protected String formatData(ReportData data) { ... }
    @Override
    protected String renderOutput(String formatted, String title) { ... }
    @Override
    protected void sendReport(String output) { ... }
}
// + ExcelReportGenerator   → 4 method override
// + HtmlReportGenerator    → 4 method override
// + EmailReportGenerator   → 4 method override
// = 16 override, 5 dosya
```

```java
var data = new ReportData("Q1 Satis Raporu", columns, rows);
new PdfReportGenerator().generate(data);
new ExcelReportGenerator().generate(data);
new HtmlReportGenerator().generate(data);
new EmailReportGenerator("yonetim@sirket.com").generate(data);
```

## Modern FP – Template Method

```java
public record ReportGenerator(
    Consumer<ReportData> dataValidator,
    Function<ReportData, String> dataFormatter,
    UnaryOperator<String> outputRenderer,
    Consumer<String> reportSender
) {
    public void generate(ReportData data) {
        dataValidator.accept(data);
        String formatted = dataFormatter.apply(data);
        String output = outputRenderer.apply(formatted);
        reportSender.accept(output);
    }
}
```

```java
public static ReportGenerator pdf() {
    return new ReportGenerator(
        data -> IO.println("  [PDF] Dogrulandi"),
        data -> {
            String headers = String.join(" | ", data.columns());
            String rows = formatRows(data, " | ");
            return headers + "\n" + "-".repeat(40) + "\n" + rows;
        },
        formatted -> "[PDF_DOCUMENT]\n" + formatted,
        output -> IO.println("  [PDF] Yazildi: rapor.pdf")
    );
}
```

```java
var data = new ReportData("Q1 Satis Raporu", columns, rows);
ReportGenerator.pdf().generate(data);
ReportGenerator.excel().generate(data);
ReportGenerator.html().generate(data);
ReportGenerator.email("yonetim@sirket.com").generate(data);
```

**Mesaj:** 4 format = 4 sinif yerine 4 factory metot. Yeni format? Yeni sinif degil, sadece 4 lambda.

# Özet: Ne Öğrendik?

# Özet: Ne Öğrendik?

**Builder**

Verbose builder → Immutability & records

# Özet: Ne Öğrendik?

**Builder**

Verbose builder → Immutability & records

**Strategy**

Interface + class → Lambda ifadesi

# Özet: Ne Öğrendik?

**Builder**

Verbose builder → Immutability & records

**Strategy**

Interface + class → Lambda ifadesi

**Decorator**

Class hiyerarşisi → andThen() kompozisyonu

# Özet: Ne Öğrendik?

**Builder**

Verbose builder → Immutability & records

**Strategy**

Interface + class → Lambda ifadesi

**Decorator**

Class hiyerarşisi → andThen() kompozisyonu

**Template Method**

Abstract class → Higher-order functions

# Özet: Ne Öğrendik?

### Builder

Verbose builder → Immutability & records

### Strategy

Interface + class → Lambda ifadesi

### Decorator

Class hiyerarşisi → andThen() kompozisyonu

### Template Method

Abstract class → Higher-order functions

### Modern Java'nın Kazanımları

- ✓ %60-80 daha az kod
- ✓ Daha yüksek okunabilirlik
- ✓ Daha kolay test edilebilirlik
- ✓ Yüksek composability – küçük davranış parçalarını (lambda, `andThen`, HOF) birleştirerek yeni akışlar kurmak

# Özet: Ne Öğrendik?

### Builder
Verbose builder → Immutability & records

### Strategy
Interface + class → Lambda ifadesi

### Decorator
Class hiyerarşisi → andThen() kompozisyonu

### Template Method
Abstract class → Higher-order functions

### Modern Java'nın Kazanımları

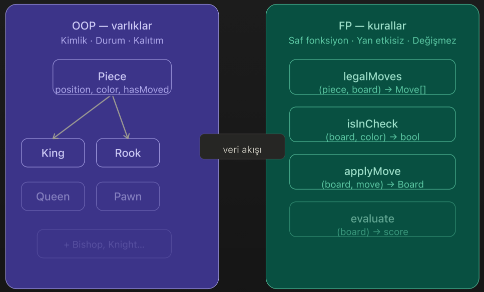
- ✓ %60-80 daha az kod
- ✓ Daha yüksek okunabilirlik
- ✓ Daha kolay test edilebilirlik
- ✓ Yüksek composability – küçük davranış parçalarını (lambda, `andThen`, HOF) birleştirerek yeni akışlar kurmak

### Kullanılan Java Özellikleri

- Lambda Expressions
- Stream API
- Records
- Pattern Matching
- Functional Interfaces
- Method References
- Sealed Classes

# OOP vs FP – karar çerçevesi

AYNI DOMAIN – IKI ROL




OOP → kimlik · durum   FP → saf fonksiyon

**OOP** “neyi” · **FP** “nasıl”

Doğru aracı doğru katmana vermek.

## OOP – kimlik ve durum

- Taşlar *nesnedir*; konum, geçmiş, “rok oldu mu?” gibi **durum** vardır.
- `King`, `Rook`, `Pawn`… `Piece` özelleşmesi – kalıtım burada doğal.
- OOP: **kimlik ve durum**.

## λ FP – kurallar ve dönüşüm

- Hamle, şah, mat: **evrensel** kurallar (kişisel tercih değil).

> Don't be a functional programmer.
>
> Don't be an object-oriented programmer.
>
> **Be a better programmer.**

Brian Goetz

*FP vs OO: Choose Two*

> *Any fool can write code that a computer can understand. Good programmers write code that humans can understand.*
>
> Martin Fowler
>
> *Refactoring: Improving the Design of Existing Code*

# Teşekkürler!

Modern Java ile Design Patterns'ın Fonksiyonel Evrimi

GitHub

`github.com/umiitkose`

GitHub QR

YouTube

`Design Patterns Serisi`

YouTube QR

Örnek Kodlar

`javadayistanbul-functional-evolution-of-design-patterns-with-modern-Java`

Repo QR

Sorularınız?